PYTHON KURSU • HAFTA 3 • GÜN 1

Listeler ve Demetler

Ders 5 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Liste oluşturup elemanlarına indeks ve dilimle ulaşabileceksiniz.
- append, insert, remove, pop ile listeyi değiştirebileceksiniz.
- sort ile sorted arasındaki farkı ve kopyalama tuzağını bileceksiniz.
- Demet (tuple) oluşturup açabileceksiniz (unpacking).
- Liste mi demet mi kullanacağınıza karar verebileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 4'in tekrarı
0:10–1:25	75 dk	Blok 1: liste oluşturma • indeks ve dilim • ekleme • silme ve arama
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: sort ve kopyalama • demetler • alışveriş listesi ve teslimat rotası • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 4'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ekranda ne yazar?

```
print(len([3, 8, 1]) + max(3, 8, 1))
```

SORU 2 • Ekranda ne yazar?

```
print(round(2.5), round(3.5))
```

SORU 3 • Liste neden sıralanmadı?

```
notlar = [5, 2, 9]
sorted(notlar)
print(notlar)  # [5, 2, 9]
```

SORU 4 • Çıktıdaki parantez ne?

```
print(divmod(130, 60))
# (2, 10)
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 2: film süresi çevirici

```
dakika = int(input("Film kaç dakika? "))
saat, kalan = divmod(dakika, 60)
print(f"{saat} saat {kalan} dakika")
```

```
ÇIKTI
Film kaç dakika? 148
2 saat 28 dakika
```

divmod iki değer döndürür; **saat, kalan = …** ikisini ayrı değişkenlere dağıtır. Bugün bunun adını koyacağız: **demet açma**.

BLOK 1 • 0:10–1:25 • 75 DK

Listeler

15 dk
Liste nedir?

20 dk
İndeks, dilim, değiştirme

20 dk
Eleman ekleme

20 dk
Silme ve arama

BLOK 1 • LISTE NEDIR? • 15 DK

Liste: sıralı ve değiştirilebilir bir değer dizisi

Benzetme: buzdolabındaki alışveriş listesi

Kâğıtta maddeler alt alta, sırayla yazılıdır. Yeni madde ekler, aldığınızı silersiniz, sırasını değiştirirsiniz. Python listesi de böyledir.

```
alisveris = ["ekmek", "süt", "yumurta"]
notlar = [78, 92, 65]
karisik = ["Ali", 34, 1.75, True]
bos = []
print(alisveris)
print(len(alisveris), len(bos))
```

```
ÇIKTI
['ekmek', 'süt', 'yumurta']
3 0
```

Başka yollarla liste elde etmek

```
harfler = list("Python")
print(harfler)
kelimeler = "elma armut kiraz".split()
print(kelimeler)
sayilar = list(range(1, 6))
print(sayilar)
print([0] * 4)
print([1, 2] + [3, 4])
```

```
ÇIKTI
['P', 'y', 't', 'h', 'o', 'n']
['elma', 'armut', 'kiraz']
[1, 2, 3, 4, 5]
[0, 0, 0, 0]
[1, 2, 3, 4]
```

BLOK 1 • İNDEKS, DILIM, DEĞIŞTIRME • 20 DK

İndeks ve dilim: string’lerde öğrendiğinizin aynısı

Eleman	"ekmek"	"süt"	"yumurta"	"peynir"	"çay"
İndeks	0	1	2	3	4
Ters indeks	-5	-4	-3	-2	-1

```
alisveris = ["ekmek", "süt", "yumurta", "peynir", "çay"]
print(alisveris[0])
print(alisveris[-1])
print(alisveris[1:3])
print(alisveris[:2])
print(alisveris[::-1])
```

```
ÇIKTI
ekmek
çay
['süt', 'yumurta']
['ekmek', 'süt']
['çay', 'peynir', 'yumurta', 'süt', 'ekmek']
```

BLOK 1 • İNDEKS, DILIM, DEĞIŞTIRME • 20 DK

Liste değiştirilebilir, string değiştirilemez

```
alisveris = ["ekmek", "süt", "yumurta"]
alisveris[1] = "ayran"
print(alisveris)
alisveris[0] = alisveris[0].upper()
print(alisveris)
```

```
ÇIKTI
['ekmek', 'ayran', 'yumurta']
['EKMEK', 'ayran', 'yumurta']
```

Benzetme: kurşun kalem ve tükenmez kalem

Liste kurşun kalemle yazılmış bir listedir: bir maddeyi silip yerine başkasını yazabilirsiniz. String tükenmezle yazılmıştır: değiştirmek için yeniden yazmanız gerekir.

```
STRING'DE AYNISI
TypeError: 'str' object does not
 support item assignment
```

İndeks hataları ve anlamları

Hatalı kod	Hata mesajı	Çözüm
`liste[3]`	`IndexError: list index out of range`	Son indeks len – 1: liste[2] ya da liste[-1]
`liste["1"]`	`TypeError: list indices must be integers or slices, not str`	İndeks tam sayı olmalı: liste[1]
`liste[3 / 2]`	`TypeError: list indices must be integers or slices, not float`	Tam bölme kullanın: liste[3 // 2]
`ad[0] = "A"`	`TypeError: 'str' object does not support item assignment`	Yeni metin oluşturun: "A" + ad[1:]

BLOK 1 • ELEMAN EKLEME • 20 DK

append, insert, extend: listeyi büyütmek

```
alisveris = ["ekmek", "süt"]
alisveris.append("yumurta")
print(alisveris)
alisveris.insert(0, "su")
print(alisveris)
alisveris.extend(["çay", "şeker"])
print(alisveris)
print(len(alisveris))
```

```
ÇIKTI
['ekmek', 'süt', 'yumurta']
['su', 'ekmek', 'süt', 'yumurta']
['su', 'ekmek', 'süt', 'yumurta', 'çay', 'şeker']
6
```

Metot	Ne yapar?	Örnek
`append(x)`	x'i SONA ekler	`liste.append("çay")`
`insert(i, x)`	x'i i. konuma yerleştirir	`liste.insert(0, "su")`
`extend(liste2)`	başka listenin elemanlarını sona ekler	`liste.extend(["a", "b"])`

BLOK 1 • ELEMAN EKLEME • 20 DK

append ile extend farkı: iç içe liste tuzağı

```
a = [1, 2]
a.append([3, 4])
print(a, len(a))
b = [1, 2]
b.extend([3, 4])
print(b, len(b))
```

```
ÇIKTI
[1, 2, [3, 4]] 3
[1, 2, 3, 4] 4
```

Benzetme: alışveriş poşeti

append, poşeti olduğu gibi sepete koyar: sepette bir poşet daha olur. extend, poşeti boşaltıp ürünleri tek tek sepete dizer.

BLOK 1 • SILME VE ARAMA • 20 DK

remove, pop, del, clear: listeden çıkarmak

```
alisveris = ["su", "ekmek", "süt", "çay", "şeker"]
alisveris.remove("süt")
print(alisveris)
son = alisveris.pop()
print(son, alisveris)
ilk = alisveris.pop(0)
print(ilk, alisveris)
del alisveris[0]
print(alisveris)
alisveris.clear()
print(alisveris)
```

```
ÇIKTI
['su', 'ekmek', 'çay', 'şeker']
şeker ['su', 'ekmek', 'çay']
su ['ekmek', 'çay']
['çay']
[]
```

BLOK 1 • SILME VE ARAMA • 20 DK

Listede arama: in, index, count

```
sepet = ["elma", "süt", "elma", "ekmek", "elma"]
print("süt" in sepet)
print("çay" in sepet)
print("çay" not in sepet)
print(sepet.index("ekmek"))
print(sepet.count("elma"))
```

```
ÇIKTI
True
False
True
3
3
```

Neden önemli?

“Listede var mı?” sorusu çok sık sorulur: sepette süt var mı, kullanıcı adı alınmış mı, bu öğrenci listede mi? Cevap True ya da False’tur; Ders 7’de if ile karar vermeye bu sonuçları kullanacağız.

Olmayan elemanı silmek ya da aramak

Hatalı kod	Hata mesajı	Çözüm
`sepet.remove("çay")`	`ValueError: list.remove(x): x not in list`	Önce kontrol: "çay" in sepet
`sepet.index("çay")`	`ValueError: 'çay' is not in list`	Önce in ile varlığını sorun
`sepet.pop()`	`IndexError: pop from empty list`	Boş listeden eleman çıkarılamaz
`sepet.append("süt", "çay")`	`TypeError: list.append() takes exactly one argument (2 given)`	Birden çok eleman için extend
`sepet.add("süt")`	`AttributeError: 'list' object has no attribute 'add'`	Listelerde append kullanılır

1:25–1:40 • 15 DK

Mola

Moladan sonra: liste = liste.sort() yazarsak listeye ne olur?

BLOK 2 • 1:40–2:55 • 75 DK

İki tuzak ve demetler

20 dk **sort ve kopyalama**	20 dk **Demetler**	15 dk **Uygulamalar**	20 dk **Alıştırmalar**

BLOK 2 • SORT VE KOPYALAMA • 20 DK

sort listeyi yerinde sıralar, sorted yeni liste verir

```
notlar = [72, 95, 64, 88]
sirali = sorted(notlar)
print(sirali, notlar)
notlar.sort()
print(notlar)
notlar.sort(reverse=True)
print(notlar)
```

```
ÇIKTI
[64, 72, 88, 95] [72, 95, 64, 88]
[64, 72, 88, 95]
[95, 88, 72, 64]
```

Tuzak

Metot listeyi yerinde değiştirip None döndürür:

```
notlar = [72, 95, 64]
notlar = notlar.sort()
print(notlar)
```

```
ÇIKTI
None
```

BLOK 2 • SORT VE KOPYALAMA • 20 DK

b = a kopyalamaz: aynı listeye ikinci isim verir

```
a = ["ekmek", "süt"]
b = a
b.append("çay")
print(a)
c = a.copy()
c.append("şeker")
print(a)
print(c)
```

```
ÇIKTI
['ekmek', 'süt', 'çay']
['ekmek', 'süt', 'çay']
['ekmek', 'süt', 'çay', 'şeker']
```

Benzetme: aynı evin iki anahtarı

b = a demek, aynı eve ikinci bir anahtar vermektir. Biri içeri girip mobilyayı değiştirirse, diğeri de değişmiş evi görür. Gerçek kopya için evin bir benzerini yapmak gerekir: a.copy().

Kopya yolları: **a.copy() · list(a) · a[:]**

BLOK 2 • SORT VE KOPYALAMA • BAŞVURU TABLOSU

Liste metotları tek bakışta

Metot	Ne yapar?	Listeyi değiştirir mi?	Ne döndürür?
`append(x)`	sona ekler	Evet	None
`insert(i, x)`	i. konuma ekler	Evet	None
`extend(l2)`	elemanları sona ekler	Evet	None
`remove(x)`	ilk x'i siler	Evet	None
`pop(i)`	konumdan çıkarır	Evet	çıkan eleman
`sort() / reverse()`	sıralar / ters çevirir	Evet	None
`index(x) / count(x)`	konum bulur / sayar	Hayır	sayı
`copy()`	kopya oluşturur	Hayır	yeni liste

BLOK 2 • DEMETLER • 20 DK

Demet (tuple): değiştirilemeyen sıralı değerler

Benzetme: kimlik kartı

Kartın üzerindeki doğum tarihi ve T.C. kimlik no basılmıştır; silip yenisini yazamazsınız. Bir arada duran, değişmemesi gereken bilgiler için demet kullanılır.

```
konum = (41.0082, 28.9784)
dogum = (15, "Mart", 1990)
print(konum[0])
print(dogum[-1])
print(len(dogum))
print(type(konum))
```

```
ÇIKTI
41.0082
1990
3
<class 'tuple'>
```

BLOK 2 • DEMETLER • 20 DK

Demet açma (unpacking): değerleri değişkenlere dağıtmak

```
konum = (41.0082, 28.9784)
enlem, boylam = konum
print("Enlem:", enlem, "Boylam:", boylam)
saat, dakika = divmod(130, 60)
print(saat, dakika)
a, b = 1, 2
a, b = b, a
print(a, b)
```

```
ÇIKTI
Enlem: 41.0082 Boylam: 28.9784
2 10
2 1
```

Tek elemanlı demet tuzağı

Parantez değil, virgül demet yapar:

```
x = (5)
y = (5,)
print(type(x), type(y))
```

```
ÇIKTI
<class 'int'> <class 'tuple'>
```

Demet hataları ve anlamları

Hatalı kod	Hata mesajı	Çözüm
`konum[0] = 40.0`	`TypeError: 'tuple' object does not support item assignment`	Yeni demet oluşturun: konum = (40.0, konum[1])
`konum.append(5)`	`AttributeError: 'tuple' object has no attribute 'append'`	Değişmesi gerekiyorsa liste kullanın
`ad, soyad = "Ayşe Nur Yılmaz".split()`	`ValueError: too many values to unpack (expected 2)`	Değişken sayısı = eleman sayısı
`x, y, z = (1, 2)`	`ValueError: not enough values to unpack (expected 3, got 2)`	Eksik değer: (1, 2, 3)

BLOK 2 • DEMETLER • 20 DK

Liste mi, demet mi?

	Liste	Demet
Yazım	[1, 2, 3]	(1, 2, 3)
Değiştirilebilir mi?	Evet	Hayır
Metotlar	11 metot (append, sort…)	2 metot (count, index)
Tipik kullanım	büyüyen/küçülen koleksiyon	sabit, bir arada duran bilgi
Örnek	sepet, yapılacaklar, notlar	koordinat, tarih, RGB rengi
Sözlükte anahtar olur mu?	Hayır	Evet (Ders 6)

Kural: **“Bu veri program çalışırken değişecek mi?”** Evet ise liste, hayır ise demet.

BLOK 2 • GERÇEK HAYATTAN UYGULAMA 1 • 8 DK

Haftalık alışveriş listesi

alisveris.py

```
alisveris = ["ekmek", "süt", "yumurta"]
alisveris.append("domates")
alisveris.insert(0, "su")
alisveris.extend(["peynir", "zeytin"])
alisveris.remove("süt")
alinan = alisveris.pop(0)
alisveris[1] = "köy yumurtası"
alisveris.sort()
print("Alınan:", alinan)
print("Kalan ürün sayısı:", len(alisveris))
print("Liste:", ", ".join(alisveris))
print("Peynir var mı?", "peynir" in alisveris)
```

ÇIKTI

```
Alınan: su
Kalan ürün sayısı: 5
Liste: domates, ekmek,
  köy yumurtası, peynir, zeytin
Peynir var mı? True
```

BLOK 2 • UYGULAMA 1 • SATIR SATIR

Liste her adımda nasıl değişti?

Adım	Liste (sonrasında)
`başlangıç`	ekmek, süt, yumurta
`append("domates")`	ekmek, süt, yumurta, domates
`insert(0, "su")`	su, ekmek, süt, yumurta, domates
`extend([…])`	su, ekmek, süt, yumurta, domates, peynir, zeytin
`remove("süt")`	su, ekmek, yumurta, domates, peynir, zeytin
`pop(0)`	ekmek, yumurta, domates, peynir, zeytin
`[1] = "köy yumurtası"`	ekmek, köy yumurtası, domates, peynir, zeytin
`sort()`	domates, ekmek, köy yumurtası, peynir, zeytin

BLOK 2 • GERÇEK HAYATTAN UYGULAMA 2 • 7 DK

Kargo teslimat rotası: demetlerden oluşan liste

rota.py

```
depo = (0, 0)
duraklar = [("Fırın", 2, 3), ("Eczane", 5, 1), ("Market", 1, 6)]
duraklar.append(("Kırtasiye", 4, 4))
duraklar.insert(0, ("Hastane", 1, 1))
ad, x, y = duraklar[0]
mesafe = abs(x - depo[0]) + abs(y - depo[1])
print(f"İlk durak: {ad} ({x}, {y}) - {mesafe} blok")
son_ad, son_x, son_y = duraklar[-1]
print(f"Son durak: {son_ad} ({son_x}, {son_y})")
print("Durak sayısı:", len(duraklar))
duraklar.reverse()
print("Dönüş rotasında ilk:", duraklar[0][0])
```

ÇIKTI

```
İlk durak: Hastane (1, 1) - 2 blok
Son durak: Kırtasiye (4, 4)
Durak sayısı: 5
Dönüş rotasında ilk: Kırtasiye
```

BLOK 2 • UYGULAMA 2 • SATIR SATIR

İki indeks art arda: önce liste, sonra demet

İfade	Sonuç
`duraklar[0]`	`("Hastane", 1, 1)`
`duraklar[0][0]`	`"Hastane"`
`duraklar[0][1]`	`1`
`duraklar[-1]`	`("Kırtasiye", 4, 4)`
`depo[0]`	`0`
`len(duraklar)`	`5`

Neden demet?

Durak eklenip çıkarılabilir (liste), ama bir durağın adı ve konumu kazara değişmemeli (demet).

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 20 DK

Alıştırma 1 ve 2

KOLAY • 4 dk

1 • Yapılacaklar listesi

Boş bir listeyle başlayın:

- “fatura öde”, “spor”, “kitap oku” ekleyin
- “spor”u silin
- En başa “su iç” ekleyin
- Listeyi ve eleman sayısını yazdırın

KOLAY-ORTA • 5 dk

2 • Sınav notları

notlar = [55, 78, 92, 40, 66] listesinde:

- Notu 40 olanı listeden çıkarın
- Yeni not 85’i ekleyin
- Listeyi büyükten küçüğe sıralayın
- En yüksek ilk üç notu dilimle yazdırın

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 20 DK

Alıştırma 3 ve 4

ORTA • 5 dk

3 • Hata avı

Kodda 3 hata var. Çalıştırın, mesajları okuyun, düzeltin.

```
renkler = ("kırmızı", "mavi")
renkler.append("yeşil")
sehirler = ["Ankara", "İzmir"]
sehirler = sehirler.sort()
print(sehirler[0])
print(sehirler[2])
```

ZOR • 6 dk

4 • Sinema koltukları

koltuklar = ["A1", "A2", "A3", "B1", "B2"]

- Müşteri “A2” ve “B1”i satın alıyor: satılanlar listesine taşıyın (pop/remove + append)
- Boş ve satılan koltuk sayılarını yazdırın
- “A3” boş mu? (True/False)
- Satılanları demete çevirip yazdırın

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1 ve 2

1 • Yapılacaklar listesi

```
yapilacaklar = []
yapilacaklar.append("fatura öde")
yapilacaklar.append("spor")
yapilacaklar.append("kitap oku")
yapilacaklar.remove("spor")
yapilacaklar.insert(0, "su iç")
print(yapilacaklar)
print(len(yapilacaklar))
```

```
['su iç', 'fatura öde', 'kitap oku'] · 3
```

2 • Sınav notları

```
notlar = [55, 78, 92, 40, 66]
notlar.remove(40)
notlar.append(85)
notlar.sort(reverse=True)
print(notlar)
print(notlar[:3])
```

```
[92, 85, 78, 66, 55]
[92, 85, 78]
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3 ve 4

3 • Hata avı (düzeltilmiş)

```
renkler = ["kırmızı", "mavi"]
renkler.append("yeşil")
sehirler = ["Ankara", "İzmir"]
sehirler.sort()
print(sehirler[0])
print(sehirler[1])
```

1. AttributeError: demet → liste
2. TypeError: None'a indeks → sort() tek başına
3. IndexError: [2] → [1]

4 • Sinema koltukları (kısaltılmış)

```
koltuklar = ["A1", "A2", "A3", "B1", "B2"]
satilanlar = []
koltuklar.remove("A2")
satilanlar.append("A2")
koltuklar.remove("B1")
satilanlar.append("B1")
print(len(koltuklar), len(satilanlar))
print("A3" in koltuklar)
print(tuple(satilanlar))
```

```
3 2 · True · ('A2', 'B1')
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- Liste: **[ ]**, sıralı ve değiştirilebilir; indeks ve dilim string'lerdeki gibi.
- Ekleme: **append, insert, extend** • Silme: **remove, pop, del, clear**.
- Arama: **in, index, count**; olmayan eleman ValueError verir.
- **sort()** None döndürür; **b = a** kopyalamaz, **a.copy()** kopyalar.
- Demet: **( )**, değiştirilemez; **x, y = konum** ile açılır.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kütüphane rafı

- 5 kitaplık bir liste oluşturun
- 2 kitap ödünç verilsin: listeden çıkarıp “ödünçte” listesine taşıyın
- Yeni gelen kitabı ekleyin, rafı alfabetik sıralayın

2 • ZORUNLU

Şehir bilgileri

3 şehri (ad, nüfus milyon, plaka) demetleri olarak bir listede tutun. İlk şehri açıp yazdırın; en kalabalık şehrin nüfusunu max ile bulun.

3 • BONUS

Sıra yönetimi

Banka sırası listesi: 2 müşteri gelsin (append), 1 müşteri işlem yapsın (pop(0)). Her adımda sırayı ve kişi sayısını yazdırın.

Teslim: ders05_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 3 • GÜN 2

Sözlükler ve kümeler

Listelerde elemanlara konumla ulaştık. Sözlüklerde isimle ulaşacağız:

- Sözlük (dict): anahtar → değer çiftleri
- Ekleme, güncelleme, silme; dict metotları
- Telefon rehberi ve ürün stok kaydı
- Küme (set): tekrarsız elemanlar

Önizleme

```
rehber = {"Ayşe": "0532 111 22 33"}
rehber["Mehmet"] = "0505 444 55 66"
print(rehber["Ayşe"])
```

```
0532 111 22 33
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
Listeler (Türkçe eğitim)	docs.python.org/tr/3/tutorial/introduction.html
Liste metotları ve demetler	docs.python.org/3/tutorial/datastructures.html
Sıralı tipler ve ortak işlemler	docs.python.org/3/library/stdtypes.html
Sıralama rehberi	docs.python.org/3/howto/sorting.html